GraphMind

A Constraint-Aware Degree Plan Generator

Project Name	GraphMind
Team Name	LogicForge
Course	CS F407 / U407 — Artificial Intelligence, Sem I 2026–27
Institution	BITS Pilani, Hyderabad Campus
Declared Topic Areas	Search & Problem Solving + Knowledge Representation & Reasoning
Declared Track	**Application**
Repository	https://github.com/VedantLiteCoder0135/LogicForge

Team Members

#	Name	ID No.	#	Name	ID No.
1	Raj Parab	2024A3PS0433H	5	Krishna Gupta	2024A3PS0380H
2	Jenil Karia	2024A7PS0100H	6	Pratham Singla	2024A8PS0486H
3	Aditya Sanadhaya	2024AAPS0263H	7	Mohammad Mubeen Mazar Khan	2024A8PS0489H
4	Kshitij Mathur	2024A8PS0462H	8	Vedant Shah	2024A7PS0135H

Project Description

GraphMind takes a student's completed coursework and generates a complete, feasible, semester-by-semester plan to graduation — satisfying prerequisite chains, corequisites, per-semester credit limits, odd/even term offerings, degree-requirement categories and minor requirements simultaneously. The problem is formulated both as a constraint satisfaction problem and as a heuristic search problem, and the two formulations are implemented and compared. Every generated plan carries a derivation trace justifying each course placement. When no valid plan exists, the system identifies the constraints responsible for infeasibility and reports possible relaxations that could restore feasibility, rather than returning an empty result.

1 Problem Statement & Motivation

We, the students at BITS Hyderabad, plan our remaining semesters manually, working from the published curriculum and a spreadsheet. The difficulty is not determining whether a single course is currently permitted — that is a direct lookup. The difficulty is that curricular decisions are coupled across time. A course offered only in odd semesters, positioned behind a two-level prerequisite chain, must be commenced by a specific term or it becomes unreachable before the graduation horizon. Introduce a minor, per-semester credit ceilings, and requirement categories that each carry an independent credit floor, and the task becomes a scheduling problem with interacting constraints — one where interacting constraints can easily be overlooked, potentially causing a requirement violation to be discovered late in the degree when fewer remedies remain. This is a planning problem, not an information-retrieval problem, and that distinction defines the project. A simple rule-based eligibility system can determine whether a student is currently eligible for a course, but determining whether all outstanding courses can be assigned to the remaining semesters while satisfying interacting constraints requires explicit search or constraint reasoning because that requires search over a combinatorial space in which each choice constrains the others.

One of the main reasons to do this project is also to help students to plan their academic life easily and avoid major issues like late graduation, lack of credits completed, or anything else related to graduation requirements

Intended users: students planning registration, evaluating whether a minor remains attainable, or assessing the impact of a failed course on their graduation timeline.

2 Connection to Course Concepts

The project uses the following techniques from the course, each in a load-bearing role rather than incidentally:

Constraint Satisfaction Problems. The planning task is formulated as a CSP in which variables are outstanding courses, domains are the remaining terms (augmented with a null value for optional courses), and constraints encode prerequisite ordering, corequisite coupling, per-term credit windows, category credit minimums, and the graduation horizon.

Backtracking search with constraint propagation. We implement chronological backtracking, augmented incrementally with **forward checking** and **arc consistency via AC-3** (Mackworth, 1977), and measure the contribution of each.

Variable and value ordering heuristics. Minimum Remaining Values with the **degree heuristic** as tiebreak for variable selection, and **Least Constraining Value** for value selection (Haralick & Elliott, 1980).

Informed state-space search. The identical problem is posed as a search over states (completed_courses, term_index) and solved with **A*** (Hart, Nilsson & Raphael, 1968).

Admissible heuristics and dominance. Two are derived: a *credit bound*, ⌈remaining_credits / max_credits_per_term⌉, admissible because no term supplies more than the ceiling; and a *critical-path bound*, the depth of the longest remaining prerequisite chain, admissible because a chain of depth k needs at least k terms under the ordering constraint. If both bounds are admissible under the final state and cost formulation, their pointwise maximum is also admissible and dominates each individual bound.

Propositional logic and entailment. Prerequisites may be Boolean formulas rather than simple flat lists — for example, 'CS F211 and (MATH F211 or MATH F213)'. These are represented as propositional formulas over course-code literals and evaluated by entailment against the completed-course set, with disjunctions creating genuine branch points for the planner.

Forward chaining to fixpoint. Requirement resolution applies derivation rules — course equivalence, category credit accumulation, prerequisite-satisfaction unlocking — until no new facts are derivable. Unlocking cascades: one course can make a chain of subsequent courses reachable.

Minimal conflict extraction. Infeasible instances are diagnosed by deletion-based minimal unsatisfiable core extraction, following the approach of Junker (2004).

Learned heuristic functions (Deep Learning component). A neural regressor is trained to predict true remaining plan length from state features, and evaluated against the hand-crafted admissible heuristics on search effort and solution optimality (Arfaee, Zilles & Holte, 2011).

3 Proposed Approach & Methodology

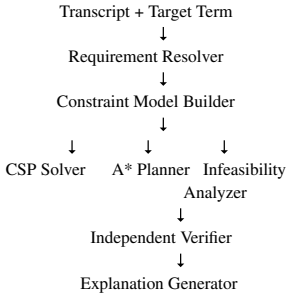
Data and Environment

Curriculum data is real. The knowledge base encodes the published BITS Pilani Hyderabad curriculum — course codes, credit structure, prerequisite formulas, requirement categories and credit thresholds, term-parity offerings — sourced from the Academic Regulations, published course handouts and course lists. We begin with a focused curriculum scope so that prerequisite, requirement, and semester constraints can be encoded and validated reliably before expanding the system. This bounded scope is a design decision: a correct, validated knowledge base for one program supports stronger results than a shallow encoding of several.

Student data is synthetic or volunteered. Development and benchmarking use authored profiles spanning representative cases — on track, behind by one course, late minor adoption, recovery from a failed prerequisite, accelerated graduation. The end-user evaluation (§3.4, E5) uses transcripts volunteered by team members and classmates for their own records. **No third-party academic records are collected or stored.**

A validation suite runs over the knowledge base from the outset, checking for prerequisite-graph cycles, undefined course codes, categories whose eligible-course credit total falls below their own minimum, and courses unreachable within the degree horizon.

Architecture



Both solvers consume the identical constraint model, which is what makes the comparison in E1 meaningful. The plan verifier is written independently of both solvers, by a team member who implemented neither, and re-validates every constraint from scratch; the system does not assess its own correctness.

Algorithms

The CSP solver performs backtracking over course-to-term assignments. Per-term credit windows are *sum* constraints spanning many variables and therefore lie outside AC-3's binary-constraint scope; these are enforced by incremental bounds checking on each assignment, and this limitation is reported rather than elided.

The search planner treats a state as a completed-course set paired with a term index, with actions corresponding to feasible course bundles for the next term. Since the bundle space is combinatorial, expansion is breadth-limited in LCV-like order, and the resulting optimality caveat is reported explicitly.

The learned heuristic is trained on self-supervised data: the optimal planner is run over several thousand randomly generated student states, yielding (state_features, true_remaining_terms) pairs. Features comprise outstanding credits per category, outstanding course count, maximum remaining chain depth, parity-locked course count, terms remaining, and unsatisfied disjunction count. Because a learned heuristic is not guaranteed to be admissible, the corresponding A* search is not guaranteed to retain the optimality guarantee of admissible A*; the frequency and magnitude of any optimality loss will be measured experimentally.

Evaluation

Metrics. Plan validity rate against the independent verifier; optimality gap against exhaustive search on small instances; search effort (nodes expanded, backtracks, wall-clock time); minimality of returned infeasibility cores, verified by brute force on small instances.

Benchmark. Approximately 200 synthetic student states, stratified by difficulty — credit-budget slack, prerequisite chain depth, minor versus no minor. Results reported as means with 95% bootstrap confidence intervals; paired solver comparisons on identical instances tested with the Wilcoxon signed-rank test, as node-count distributions are skewed.

ID	Experiment	Independent variable	Measures
E1	Solver comparison	CSP backtracking vs. A*	Validity, optimality, time, nodes
E2	Propagation ablation	plain BT → +forward checking → +AC-3 → +MRV/LCV	Backtracks, runtime
E3	Heuristic ablation	credit bound/critical path/composite / learned	Nodes expanded, optimality gap
E4	Grounding comparison	Correctness validation/ Independent verifier	Plan validity and constraint violations
E5	End-user evaluation	6–10 volunteer students, own transcripts	Plan correctness, usefulness rating

E4 validates generated plans using the independent verifier, which checks each prerequisite, requirement, credit, and semester constraint without relying on either planning algorithm. E5 addresses the Application track's requirement for realistic users.

4 Related Work

Degree audit systems. Commercial degree-audit software (for example, Ellucian DegreeWorks) is widely deployed and performs requirement tracking and eligibility checking against an encoded catalogue. Such systems report *progress* and *eligibility*; they are not search systems, do not generate complete multi-semester schedules under interacting constraints, and do not diagnose infeasibility. GraphMind's contribution relative to this class is plan *generation* and *infeasibility explanation*.

Educational timetabling as constraint satisfaction. Automated timetabling is a mature CSP application area, surveyed by Schaerf (1999) and sustained by the International Timetabling Competition series. That literature addresses the *institution's* problem — assigning courses to rooms, slots,s and instructors under resource conflicts. GraphMind addresses the complementary *student-side* problem: assigning one student's outstanding courses to semesters under prerequisite ordering, credit windows and requirement categories. The formulation machinery is shared; the problem and constraint structure are not.

Explanation for over-constrained problems. Junker (2004) introduced QUICKXPLAIN for computing preferred explanations and relaxations when a constraint problem admits no solution. Our infeasibility analyser applies deletion-based minimal-core extraction in this tradition, specialised to curricular constraints and rendered in natural language.

Learned heuristics for search. Arfaee, Zilles & Holte (2011) show that heuristic functions for large state spaces can be learned to improve search efficiency at the cost of admissibility. We apply this to curricular planning and report the efficiency-versus-optimality trade-off rather than assuming a favourable outcome.

5 Feasibility

Week-by-Week Milestones

Week	Dates	Milestone
1	3–9 Oct	KB schema, loader, validation suite; 10-course toy curriculum committed
2	10–16 Oct	Prerequisite formulas and entailment evaluator; fixpoint requirement closure
3	17–23 Oct	Real curriculum encoded and cross-checked; CSP model constructed
4	24–30 Oct	Backtracking solver producing valid plans; independent verifier
5	31 Oct – 6 Nov	End-to-end GraphMind system: transcript → constraints → valid semester plan → verification

Week	Dates	Milestone
6	7–13 Nov	Forward checking, AC-3, MRV/LCV; propagation ablation experiment
7	14–20 Nov	A* planner with admissible heuristics; infeasibility analyser; solver comparison
8	21–24 Nov	Learned heuristic integration; complete all core features; full system testing
9	25-27 Nov	Benchmark experiments, user evaluation, explanation interface, and final bug fixing
10	28-30 Nov	Final demo, report completion, presentation preparation, and submission

The Week 5 milestone is deliberately a complete working system. All subsequent work improves a system that already functions end to end, which bounds integration risk. The system is designed to be feature-complete by Week 8, leaving the final weeks for benchmarking, user evaluation, polishing, and the final demonstration by the end of November.

Role Assignment

Work is organised as four pairs, each owning a component end to end — implementation, testing, and the ability to explain it. Pairing ensures at least two members understand each subsystem.

Pair	Members	Ownership
A	Krishna Gupta, Pratham Singla	Curriculum KB and validation; prerequisite formulas, entailment evaluator, forward-chaining closure
B	Aditya Sanadhaya, Kshitij Mathur	CSP formulation; backtracking with forward checking, AC-3, MRV/LCV; E2
C	Raj Parab, Jenil Karia	A* planner, admissible heuristics; learned heuristic; E1, E3
D	Mubeen Khan, Vedant Shah	Plan verifier; infeasibility analyser; benchmark and statistics; integration and interface

Pair A is on the critical path and begins immediately; the Week 1 toy curriculum lets Pairs B, C and D develop in parallel without blocking. The verifier is owned by Pair D, which implements neither solver, preserving independence of validation.

Risks

The principal risk is curriculum encoding overrunning and blocking downstream work, mitigated by dedicated ownership from Week 1 and the parallel toy curriculum. Secondary risks are the combinatorial bundle-expansion factor in the search planner — mitigated by breadth limiting with the caveat reported, and by the CSP solver as a complete alternative — and the possibility that the learned heuristic does not outperform the hand-crafted one, which we treat as a reportable result rather than a failure.

6 References

1. Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson. — Ch. 3, 6, 7–9.
2. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
3. Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, 8(1), 99–118.
4. Haralick, R. M. & Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3), 263–313.
5. Schaerf, A. (1999). A survey of automated timetabling. *Artificial Intelligence Review*, 13(2), 87–127. https://link.springer.com/article/10.1023/A:1006576209967
6. Junker, U. (2004). QUICKXPLAIN: Preferred explanations and relaxations for over-constrained problems. *Proceedings of AAAI 2004*.
7. Arfaee, S. J., Zilles, S., & Holte, R. C. (2011). Learning heuristic functions for large state spaces. *Artificial Intelligence*, 175(16–17), 2075–2098.
8. BITS Pilani. *Academic Regulations* and published course handouts, Hyderabad Campus (primary source for the knowledge base); CS F407/U407 Project Guidelines, Appendices A and B.